

Дискретно-событийное моделирование

7.1 Модель M/M/c

Модель M/M/c (по классификации Кендалла) — это система массового обслуживания со следующими свойствами:

- M (Markovian) — входящий поток заявок пуассоновский, интервалы между прибытиями распределены экспоненциально с параметром λ .
- M — время обслуживания каждой заявки распределено экспоненциально с параметром μ .
- c — количество идентичных обслуживающих приборов (каналов), работающих параллельно.

7.1.1 Основные параметры

Параметр	Обозначение	Описание
Интенсивность входящего потока	λ	Среднее число заявок в единицу времени. Интервалы прибытия $\sim \text{Exp}(1/\lambda)$
Интенсивность обслуживания (одного канала)	μ	Среднее число заявок, обслуживаемых одним прибором в единицу времени. Длительность обслуживания $\sim \text{Exp}(1/\mu)$

Параметр	Обозначение	Описание
Число каналов	c	Количество параллельных серверов
Загрузка системы на один канал (traffic intensity)	$\rho = \lambda / (c \cdot \mu)$	Условие стационарности: $\rho < 1$

7.1.2 Основные аналитические характеристики (стационарные)

- Вероятность простоя всех каналов:

$$P_0 = \left[\sum_{n=0}^{c-1} \frac{(c\rho)^n}{n!} + \frac{(c\rho)^c}{c!(1-\rho)} \right]^{-1}.$$

- Вероятность того, что прибывшая заявка будет ждать (формула Эрланга):

$$P_{\text{wait}} = \frac{(c\rho)^c}{c!(1-\rho)} P_0.$$

- Среднее число заявок в очереди:

$$L_q = \frac{\rho}{(1-\rho)} P_{\text{wait}}.$$

- Среднее время ожидания в очереди, среднее время пребывания в системе и среднее число заявок в системе определяются по формулам Литтла:

$$W_q = \frac{L_q}{\lambda}, \quad W = W_q + \frac{1}{\mu}, \quad L = \lambda W.$$

7.1.3 Отличия от модели М/М/1

Характеристика	М/М/1	М/М/с
Число каналов	1	с ($c > 1$)
Поведение заявки	Ждёт единственный канал	Ждёт любой свободный канал из с
Условие стационарности	$\lambda < \mu$	$\lambda < c\mu$
Формулы для W_q	$\frac{\rho}{\mu(1-\rho)}$	Более сложные
Вероятность ожидания	Равна ρ	Определяется формулой Эрланга

7.1.4 Реализация

```

using StableRNGs
using Distributions
using ConcurrentSim
using ResumableFunctions

#set simulation parameters
rng = StableRNG(123)
num_customers = 10 # total number of customers generated

# set queue parameters
num_servers = 2 # number of servers
mu = 1.0 / 2 # service rate
lam = 0.9 # arrival rate
arrival_dist = Exponential(1 / lam) # interarrival time distribution
service_dist = Exponential(1 / mu) # service time distribution

# define customer behavior
@resumable function customer(
    env::Environment,
    server::Resource,
    id::Integer,
    t_a::Float64,
    d_s::Distribution,
)
    @yield timeout(env, t_a) # customer arrives
    println("Customer $id arrived: ", now(env))
    @yield request(server) # customer starts service
    println("Customer $id entered service: ", now(env))

```

```
@yield timeout(env, rand(rng, d_s)) # server is busy
@yield unlock(server) # customer exits service
println("Customer $id exited service: ", now(env))
end

# setup and run simulation
function setup_and_run()
    sim = Simulation() # initialize simulation environment
    server = Resource(sim, num_servers) # initialize servers
    arrival_time = 0.0
    for i = 1:num_customers # initialize customers
        arrival_time += rand(rng, arrival_dist)
        @process customer(sim, server, i, arrival_time, service_dist)
    end
    run(sim) # run simulation
end
setup_and_run()
```

7.1.5 Задание

- Переведите в структуру DrWatson.
- Добавьте необходимые графики.

7.2 Модель Росса

Модель представляет собой классический пример системы массового обслуживания с конечной популяцией, резервом и ремонтом.

7.2.1 Формальная постановка задачи

- В системе находятся N идентичных машин, которые постоянно работают и могут выходить из строя.
- S машин находятся в резерве и готовы немедленно заменить любую отказавшую.
- Одно ремонтное устройство (ремонтник), которое может одновременно ремонтировать только одну машину.

- Когда работающая машина ломается, происходит следующее:
 - Немедленно берётся одна резервная машина (если она есть) и запускается в работу вместо сломавшейся.
 - Сломанная машина отправляется в ремонт.
 - Если резерва нет, система падает (crash). Моделирование заканчивается.
- После ремонта машина пополняет пул резервных (становится исправной и ждёт).
- Требуется оценить среднее время до падения системы $E[T]$ при заданных распределениях наработки до отказа и времени ремонта.

7.2.2 Марковская модель

Данная система может быть описана как марковский процесс с непрерывным временем и конечным числом состояний. Состояние можно определить как число работоспособных машин (работающих плюс резервных) или число машин в ремонте.

Пусть k — число исправных машин, $0 \leq k \leq N + S$. Классический подход из учебника Росса: состояние — число исправных машин, а число работающих всегда равно N , если есть хотя бы N исправных.

Состояния:

- i — число исправных машин, $i = 0, 1, \dots, N + S$.
- Число работающих машин всегда $\min(N, i)$.
- Число резервных машин — $\max(0, i - N)$.
- Число машин в ремонте — $N + S - i$.

Интенсивности переходов:

- При отказе машины (если есть работающая) число исправных машин уменьшается на 1. Интенсивность: $\min(N, i)\lambda$ (если $i \geq 1$).
- При завершении ремонта число исправных увеличивается на 1. Интенсивность: μ , если есть хотя бы одна машина в ремонте (т.е. $i < N + S$). Ремонтник один, поэтому интенсивность постоянна, когда очередь на ремонт не пуста.

Система падает, когда число исправных машин $i = N - 1$ (если ещё есть работающая) или $i = N$. Падение происходит в момент отказа, когда нет резерва. Это соответствует переходу из состояния N (0 резерва, все N машин работают) в неработоспособное состояние. В момент отказа одной из N машин при $i = N$

запасных нет, следовательно, система падает. Значит, падение наступает при первом достижении состояния $i = N - 1$ (при переходе $i = N \rightarrow N - 1$).

Аналитически среднее время до отказа можно найти, решив систему линейных уравнений для средних времен достижения поглощающего состояния.

7.2.3 Реализация

Параметры по умолчанию:

- $N = 10$ — основные машины;
- $S = 3$ — запасные машины;
- Нарботка до отказа одной машины $\sim \exp(\lambda)$, где $\lambda = 100$ (среднее время безотказной работы 100 часов);
- Время ремонта $\sim \exp(\mu)$, где $\mu = 1$ (среднее время ремонта 1 час).
- Машин много и они ломаются часто.
- Интенсивность отказов одной машины $1/100 = 0.01$, но так как их $N = 10$, суммарная интенсивность отказов работающих машин равна $N \cdot 0.01 = 0.1$ отказов в час.
- Ремонт относительно медленный (1 час на ремонт), поэтому резерв быстро истощается.
- В среднем каждые 10 часов ломается одна машина, за это время ремонтник успевает починить 10 машин, но начальный запас всего 3.
- При случайных флуктуациях быстро возникает ситуация, когда резерв исчерпывается.
- Увеличив S (число запасных), мы получаем значительно большее время падения.
- Например, $S = 5$ может дать сотни часов.
- Это иллюстрирует важность резервирования.

```
using ResumableFunctions
```

```
using ConcurrentSim
```

```
using Distributions
```

```
using Random
```

```
using StableRNGs
```

```
const RUNS = 5
```

```
const N = 10
```

```
const S = 3
```

```
const SEED = 150
```

```

const LAMBDA = 100
const MU = 1

const rng = StableRNG(42) # setting a random seed for reproducibility
const F = Exponential(LAMBDA)
const G = Exponential(MU)

@resumable function machine(
    env::Environment,
    repair_facility::Resource,
    spares::Store{Process},
)
    while true
        try
            @yield timeout(env, Inf)
        catch
        end
        @yield timeout(env, rand(rng, F))
        get_spare = take!(spares)
        @yield get_spare | timeout(env)
        if state(get_spare) != ConcurrentSim.idle
            @yield interrupt(value(get_spare))
        else
            throw(StopSimulation("No more spares!"))
        end
        @yield request(repair_facility)
        @yield timeout(env, rand(rng, G))
        @yield unlock(repair_facility)
        @yield put!(spares, active_process(env))
    end
end

@resumable function start_sim(
    env::Environment,
    repair_facility::Resource,
    spares::Store{Process},
)
    for i = 1:N
        proc = @process machine(env, repair_facility, spares)
        @yield interrupt(proc)
    end
    for i = 1:S
        proc = @process machine(env, repair_facility, spares)
        @yield put!(spares, proc)
    end
end

```

```
    end
end

function sim_repair()
    sim = Simulation()
    repair_facility = Resource(sim)
    spares = Store{Process}(sim)
    @process start_sim(sim, repair_facility, spares)
    msg = run(sim)
    stop_time = now(sim)
    println("At time $stop_time: $msg")
    stop_time
end

results = Float64[]
for i = 1:RUNS
    push!(results, sim_repair())
end
println("Average crash time: ", sum(results)/RUNS)
```

7.2.4 Задание

- Переведите в структуру DrWatson.
- Добавьте нескольких ремонтников.
- Сделайте прогон для разного количества машин.
- Проведите мониторинг загрузки ремонтника, средней длины очереди на ремонт.
- Построение графиков изменения числа исправных машин во времени.
- Сравните с аналитическим решением.

7.3 Задание

- Создать рабочий каталог для кода.
- Установить необходимые пакеты.
- Выполнить предложенный код.
- Преобразовать код в литературный стиль.
- Сгенерировать из литературного кода:
 - чистый код;

- jupyter notebook;
- документацию в формате Quarto.
- Выполнить код из jupyter notebook.
- Интегрировать документацию в формате Quarto в отчёт.
- Добавить в код в литературном стиле вычисление для набора параметров.
- Сгенерировать из литературного кода с параметрами:
 - чистый код;
 - jupyter notebook;
 - документацию в формате Quarto.
- Выполнить код из jupyter notebook с параметрами.
- Интегрировать документацию с параметрами в формате Quarto в отчёт.

